

GLA UNIVERSITY



MINI PROJECT SYNOPSIS ON “EVENTHIVE - EVENT BOOKING PLATFORM”

Submitted By

Biprendra Kumar (2415000461)
Aditya Yadav (2415000124)
Jitendra Chauhan (2415000735)
Manisha Sharma (2415000938)
Amritanjai Singh (2415000197)

Submitted To

Faculty Name: Mr. Akash Kumar Choudhary
Master Trainer

DECLARATION

We hereby declare that the project entitled “**EventHive - Event Booking Platform**” is our proposed mini project work developed as part of our academic curriculum.

The project is designed to provide a digital event-discovery, event-package and booking-management solution. The proposed system will allow clients to explore event packages, submit booking requirements and receive booking status information, while administrators manage packages, event information and booking enquiries.

We declare that the planning, design, development, testing and documentation of this project will be carried out by the following team members under the guidance of **Mr. Akash Kumar Choudhary, Master Trainer**.

1. Biprendra Kumar - 2415000461
2. Aditya Yadav - 2415000124
3. Jitendra Chauhan - 2415000735
4. Manisha Sharma - 2415000938
5. Amritanjai Singh - 2415000197

INDEX

1	Introduction	4
2	Primary Reason to Choose This Project	5
3	The Main Objective of the Project	5
4	Scope of the Project	6
5	Working Methodology of the Project	7
6	Details About the Hardware and Software	8
7	Testing Technology	8
8	Frontend and Backend	9
9	Module Description	10
10	Data Flow Diagram - 0 Level, 1 Level and 2 Level	11-13
11	References	14

INTRODUCTION

EventHive is a web-based event booking platform designed for an event-management startup that needs a clear online presence for showcasing event packages and capturing qualified client bookings or leads. Event planning often begins with scattered information, repeated phone calls and delayed responses. Clients may not be able to compare available packages, understand inclusions or communicate their requirements in an organized manner.

The proposed platform brings event discovery and booking management into one responsive application. It is developed using the MERN stack - MongoDB, Express.js, React.js and Node.js - to support a modern user interface, REST-based services and flexible storage of event, package and booking information.

ABOUT THE PROJECT

Through EventHive, a visitor can browse event categories and package listings, review package details and submit an enquiry or booking request with event preferences. The platform captures essential details such as the selected package, event date, location, attendee estimate and contact information. An administrator can add and update packages, review booking requests and maintain the event catalogue. This approach improves visibility for clients and gives the startup an organized record of leads and bookings.

Project Title: **“EventHive - Event Booking Platform using MERN Stack”**

PRIMARY REASON TO CHOOSE THIS PROJECT

Event planning services depend on fast communication and well-presented package information. A client may be interested in a wedding, birthday, corporate event or other celebration, but may not find complete package details online. Manual enquiries can be difficult to track and can lead to delayed follow-up, duplicate information and inconsistent booking records.

EventHive is proposed as a centralized digital solution for presenting event packages and collecting booking requests in a structured way. The major reasons for choosing this project are:

- A clear online showcase for event packages and services.
- Convenient package browsing for prospective clients.
- Structured capture of event requirements and booking enquiries.
- Reduced dependence on unorganized phone or message records.
- Improved follow-up through a centralized booking list.
- Easy management of package details by administrators.
- Better visibility of client preferences, dates and locations.
- A practical full-stack use case for REST APIs, role-based access and MongoDB data management.

THE MAIN OBJECTIVE OF THE PROJECT

The main objective of EventHive is to develop a secure, centralized and user-friendly event booking platform. The system aims to:

- Allow clients to explore event categories and available packages.
- Show package descriptions, inclusions and relevant event information.
- Enable clients to submit booking enquiries with required event details.
- Store and organize client booking requests.
- Allow administrators to create, update and manage packages.
- Help administrators review and update booking status.
- Validate required information before an enquiry is submitted.
- Provide a responsive experience across common desktop and mobile browsers.

SCOPE OF THE PROJECT

The scope of EventHive covers digital presentation of event packages and structured management of booking enquiries for an event-management startup.

Client Scope

Clients will be able to:

- Browse event categories and package listings.
- View package descriptions, inclusions and service information.
- Select a package and provide event requirements.
- Submit contact details, preferred date, venue/location and attendee estimate.
- Receive acknowledgement or booking-status information.

Administrator Scope

Administrators will be able to:

- Log in to the administration area.
- Add, edit and remove event package information.
- Manage package descriptions and displayed details.
- View booking enquiries and client requirements.
- Update the progress or status of a booking request.
- Maintain organized records for follow-up.

The initial project focuses on web-based event presentation and enquiry/booking management. Payment processing, vendor marketplace integration and live external calendar synchronization are outside the current scope unless they are separately approved.

WORKING METHODOLOGY OF THE PROJECT



The development follows an iterative MERN workflow. The team first identifies the booking journey and defines user roles, event information, package details, booking records and enquiry fields. Responsive React screens are then connected to Express REST APIs. MongoDB stores the structured data, and each completed feature is tested with valid, invalid and boundary inputs before it is reviewed with the team.

DETAILS ABOUT THE HARDWARE AND THE SOFTWARE

System Requirements

Supported Operating System: Windows 10 / Windows 11, Linux or macOS.

Software Required: Visual Studio Code, Node.js, npm, MongoDB, Git and GitHub, a modern web browser, Postman, React.js and Express.js.

Hardware Requirements

A basic computer system is sufficient for development and deployment testing. Recommended minimum configuration:

- Processor: Dual Core or better.
- RAM: 4 GB minimum, 8 GB recommended.
- Storage: At least 10 GB free space.
- Stable internet connection.
- Keyboard, mouse and monitor/display.

LISTING OUT TESTING TECHNOLOGY

- Postman - API and backend endpoint testing.
- Browser Developer Tools - frontend debugging and responsive-interface inspection.
- Manual functional testing - package browsing, enquiry submission and booking management workflows.
- Validation testing - missing fields, invalid dates and incorrect contact inputs.
- Role-based testing - verifying that administrator actions are protected from general visitors.

FRONTEND AND BACKEND

Frontend

The frontend of EventHive will be developed using HTML, CSS, JavaScript, React.js, React Router and Tailwind CSS. It will include a home page, event category views, package listing and detail pages, a booking/enquiry form, acknowledgement view and an administrator dashboard. The frontend is responsible for clear navigation, responsive presentation and client-side validation.

Backend

The backend of EventHive will be developed using Node.js, Express.js, MongoDB, Mongoose, JWT authentication, bcryptjs and REST APIs. The backend will manage user authentication, event packages, client enquiries, booking records, status updates and role-based administrator access. MongoDB collections will store structured data for users, packages and booking requests.

Application Architecture

React communicates with the Express API through well-defined REST endpoints. The API validates incoming data, applies access rules and persists approved records in MongoDB. This separation keeps user-interface concerns independent from business logic, making the platform easier to test, extend and maintain.

Quality and Security Considerations

Administrative routes are protected through authenticated access, passwords are stored using secure hashing and server-side validation is applied before booking records are created. The interface is designed to remain usable on common desktop and mobile browsers.

MODULE DESCRIPTION

1. Client and Enquiry Module

Allows visitors to browse event offerings, select a package and submit the information required for an event booking enquiry.

2. Event Package Module

Maintains package information such as title, category, description, inclusions, images and displayed availability details.

3. Booking Management Module

Creates and stores booking records containing selected package, client contact details, event date, location, attendee estimate and booking status.

4. Administrator Module

Provides secure administration functions for managing package data and reviewing or updating booking enquiries.

5. Authentication and Access Module

Protects administrative routes through login credentials, password hashing and JWT-based authorization.

6. Validation and Status Module

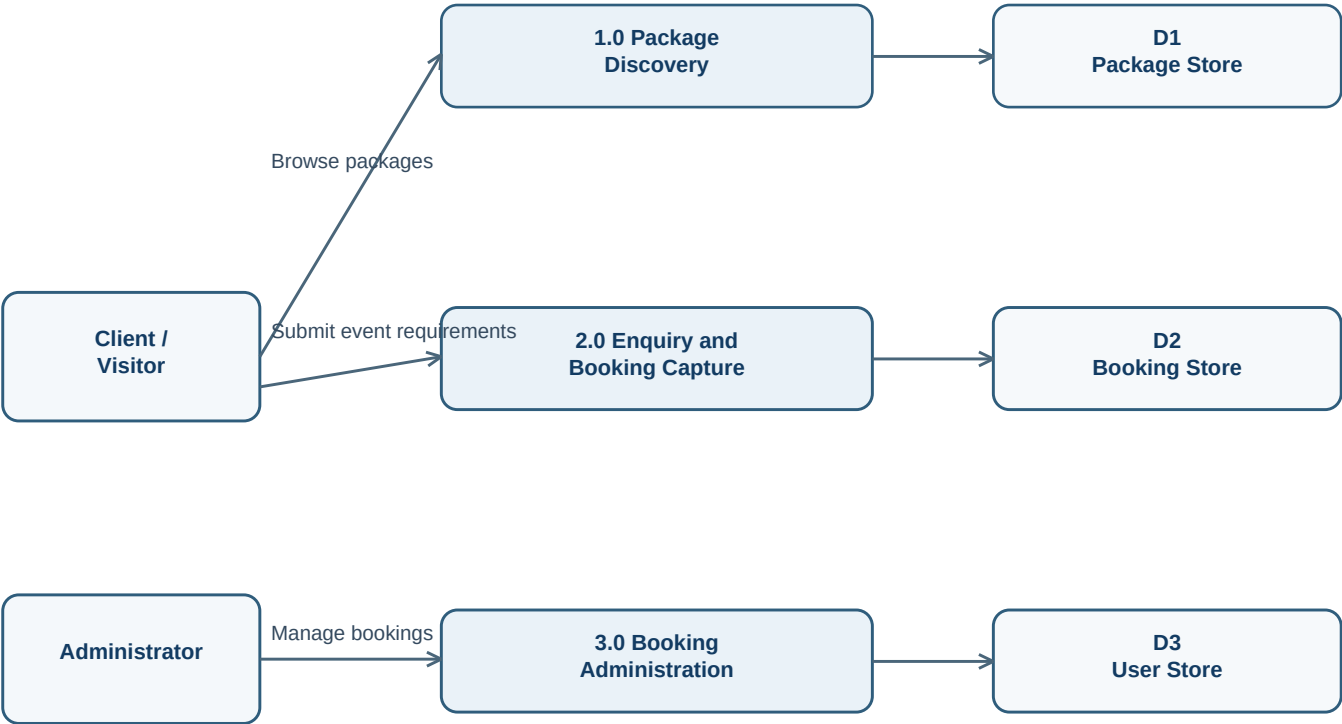
Checks required fields, supports clear booking-status updates and helps ensure that stored booking data is complete and usable.

DATA FLOW DIAGRAMS

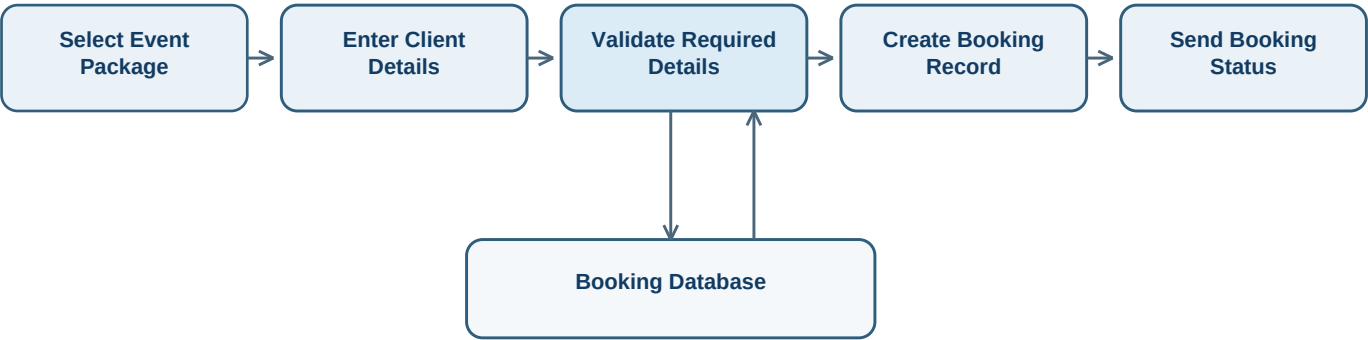
0 LEVEL DFD - CONTEXT DIAGRAM



1 LEVEL DFD



2 LEVEL DFD - BOOKING AND LEAD PROCESSING



BIBLIOGRAPHY

The project development will refer to official documentation and learning resources related to the MERN stack, web development and database management.

References

1. React Documentation

<https://react.dev/>

Used for React components, hooks and frontend development.

2. Node.js Documentation

<https://nodejs.org/docs/latest/api/>

Used for backend runtime and server-side JavaScript development.

3. Express.js Documentation

<https://expressjs.com/>

Used for REST APIs, routing and middleware implementation.

4. MongoDB Documentation

<https://www.mongodb.com/docs/>

Used for database design, MongoDB operations and data management.

5. Mongoose Documentation

<https://mongoosejs.com/docs/>

Used for MongoDB object modeling and schema design.